Chapter 5 — Linear and Logistic Regression

Linear and logistic regression are ML's "hello world" and still everywhere in production: as **baselines** (simple reference models fancier ones must beat), as the final layer of neural networks, in click-through-rate models (share of shown items clicked), credit scores and A/B-test analyses (experiments comparing two versions on random user groups). They are also the clearest place to learn the machinery every model uses: a **model**, a **loss function**, and an **optimizer** (gradient descent) that lowers the loss.

What you'll learn: the supervised workflow; linear regression fitted by the normal equation and by gradient descent (NumPy vs scikit-learn); GD variants and feature scaling; assumptions and residual plots; polynomial regression and under/overfitting; Ridge, Lasso, ElasticNet; logistic regression from scratch; one-vs-rest and softmax; interpreting coefficients.

All code is in `code/` (Python 3.14, scikit-learn 1.9, NumPy 2.5). Outputs are real; timings vary by machine.

5.1 The supervised-learning workflow

What it is

Supervised learning learns from **labeled data** (examples with the correct answer attached) and predicts answers for **unseen** examples. **Regression** predicts a number; **classification** a category. (Overview: Chapter 1.)

How it works — the steps

1. **Get data** — public datasets (Kaggle, the UCI repository, `sklearn.datasets`) or your own: CSV, Excel, database tables, text.
2. **Inspect and clean** — drop mostly-empty unimportant rows/columns or **impute** (fill in) missing values with mean, median, mode or a default; some models refuse NaNs.
3. **Encode categories** — **label (integer) encoding** (Red → 0, Blue → 1, Green → 2) is fine for trees but tells a linear model Green > Blue > Red; **one-hot encoding** (`is_Red`, `is_Blue`, `is_Green`) avoids the fake order (all encoders: Chapter 4).
4. **Split** into train and test (80/20 or 70/30).
5. **Choose and fit** a model on the training set.
6. **Evaluate** on test with a suitable metric against a trivial baseline (e.g. always predict the average).
7. **Iterate** on features, model, **hyperparameters** (settings chosen before training, e.g. tree depth; Chapter 6).

Choosing between linear regression and a decision tree

	Linear regression	Decision tree
Task	Regression (logistic regression for classification)	Regression and classification
How it works	Fits a line / plane / **hyperplane** (flat surface in many dimensions) minimising squared error	Splits data with if-then questions into groups with similar outcomes
Strengths	Simple, fast, interpretable coefficients, **extrapolates** a trend beyond the data	Non-linear patterns and **interactions** (one feature's effect depending on another), mixed feature types, easy to visualise
Weaknesses	Misses non-linearity without engineered features	**Overfits** (memorises noise) if too deep; can't extrapolate
Pick it when	Roughly linear (weight vs height)	Clear thresholds or if-then structure

(Trees in depth: Chapter 7.)

The workflow in code

Predict **petal length** from the other iris measurements plus one-hot species:

```python
# code/supervised_walkthrough.py
iris = load_iris(as_frame=True)                          # bundled with sklearn, no download
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = iris.target_names[df.pop("target")]
print(df.head(3).to_string(), "\n")
print("missing values (total):", df.isna().sum().sum())

X = pd.get_dummies(df.drop(columns="petal_length"), columns=["species"], dtype=int)
y = df["petal_length"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)   # 80/20

lin = LinearRegression().fit(X_tr, y_tr)
print("coefficients:", {c: round(float(w), 3) for c, w in zip(X.columns, lin.coef_)})
print("intercept:", round(float(lin.intercept_), 3))
for name, m in [("linear regression", lin),
                ("tree, no depth limit", DecisionTreeRegressor(random_state=0)),
                ("tree, max_depth=3", DecisionTreeRegressor(max_depth=3, random_state=0))]:
    m.fit(X_tr, y_tr)
    print(name, mean_squared_error(y_tr, m.predict(X_tr)), mean_squared_error(y_te, m.predict(X_te)))
```

Output:

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa

missing values (total): 0
model inputs: ['sepal_length', 'sepal_width', 'petal_width', 'species_setosa', 'species_versicolor',
'species_virginica']

coefficients: {'sepal_length': 0.585, 'sepal_width': -0.145, 'petal_width': 0.64, 'species_setosa': -1.144,
'species_versicolor': 0.315, 'species_virginica': 0.829}
intercept: 0.019

y range: 1.0 to 6.9 cm, std 1.77
model                        train MSE  test MSE
baseline (predict mean)                    3.302
linear regression                0.061     0.088
tree, no depth limit             0.001     0.142
tree, max_depth=3                0.078     0.121
```

Interpreting the result. **MSE** (mean squared error, in cm²) only means something against the target's scale and a baseline: predicting the mean gives 3.3, so 0.088 (≈ √0.088 ≈ 0.3 cm typical error) is good; MSE 5 would be worse than guessing the mean. The unlimited tree scores ~0 on training but worse on test: it **overfit**. Limiting `max_depth` (or raising `min_samples_split`) trades training accuracy for **generalisation** (performance on unseen data).

The three species columns always sum to 1, duplicating the intercept (the **dummy-variable trap**). `LinearRegression` still works — it returns the **minimum-norm solution** (of all equally good weight sets, the smallest), which is why the species coefficients sum to 0 — but they are only meaningful relative to each other. Use `drop="first"` (`OneHotEncoder`) or `drop_first=True` (`get_dummies`) to read each against a reference species.

Self-check

1. Train MSE 0.001, test 0.142? → Overfitting.
2. Why one-hot rather than integer-encode `species` for linear regression? → Integers impose fake order and equal spacing.

TL;DR

Get data → clean → encode → split → fit → evaluate against a baseline → iterate. Linear models for roughly linear trends; depth-limited trees for thresholds and interactions.

5.2 Linear regression: model and loss

What it is

Linear regression predicts a number as a weighted sum of features plus a constant:

$\hat{y} = w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + b = \mathbf{w \cdot x + b}$

- $\hat{y}$ ("y-hat") — prediction; x — the **feature vector** of one example.
- w — **weights** (coefficients): change in $\hat{y}$ when that feature rises by 1, others fixed.
- b — **bias** / **intercept**: the prediction when all features are 0.

One feature → line; two → plane; more → hyperplane. Trick used in all the maths: add a constant feature $x_0 = 1$ so b becomes w_0 and $\hat{y} = \theta^T x$ (θ = all parameters).

The loss: mean squared error

For m examples: $\text{MSE}(\theta) = (1/m)\ \Sigma_i\ (\hat{y}_i - y_i)^2$

Worked example: $y = (3, 5, 7)$, $\hat{y} = (2.5, 5, 8)$: errors $(-0.5, 0, 1)$, squared $(0.25, 0, 1)$, MSE = **0.417**; **RMSE** = $\sqrt{0.417} = 0.65$ (target units).

Why squares? Errors become positive, big errors are punished more, and the loss is a smooth **convex** bowl with one minimum calculus can find. Minimising MSE is also **maximum likelihood** (parameters under which the data is most probable) when noise is Gaussian.

Self-check

1. Errors $(1, -1, 2)$: MSE? → 2.

TL;DR

$\hat{y} = w \cdot x + b$; fit by minimising average squared error — a convex bowl with one minimum.

5.3 Fitting it: the normal equation vs gradient descent

What it is

1. **Normal equation** — solve for θ directly with linear algebra (read the lowest point off a map).

2. **Gradient descent (GD)** — repeatedly step downhill (walk down in fog by feeling the slope). Fundamentals: Chapter 2, section 2.5.

How it works

Normal equation. At the bottom the gradient $(2/m)\ X^T(X\theta - y)$ is zero, giving $X^TX\theta = X^Ty$; if X^TX is invertible:

$$\theta = (X^TX)^{-1}\ X^Ty$$

X is the $m \times (d+1)$ matrix (with the column of 1s), y the targets. Never compute the inverse: solve $X^TX\theta = X^Ty$ (`np.linalg.solve`) or use least squares (`np.linalg.lstsq`, which uses the SVD and also works when X^TX is singular). scikit-learn's `LinearRegression` calls SciPy's least-squares solver.

Gradient descent. $\nabla \mathrm{MSE}(\theta) = (2/m)\ X^T(X\theta - y)$. Why: by the **chain rule** (multiply the slopes of nested functions), the derivative of one term $(\theta^Tx_i - y_i)^2$ w.r.t. θ_j is $2(\theta^Tx_i - y_i)\cdot x_{ij}$ — "twice the error times that feature"; average and stack. Update $\theta \leftarrow \theta - \eta\ \nabla \mathrm{MSE}(\theta)$ (η = **learning rate**); a full pass over the data is an **epoch**.

One step by hand: $x = (1, 2)$, $y = (2, 4)$, $w = b = 0$, $\eta = 0.1$: errors $(-2, -4)$; $\partial/\partial b = -6$, $\partial/\partial w = (1\cdot-2 + 2\cdot-4) = -10$; new $b = 0.6$, $w = 1.0$ — heading toward $y = 2x$.

```python
# code/linreg_from_scratch.py (key parts)
def add_bias(X):
    return np.c_[np.ones(len(X)), X]                    # column of ones -> intercept is w0

def normal_equation(X, y):
    Xb = add_bias(X)
    return np.linalg.solve(Xb.T @ Xb, Xb.T @ y)         # solves (X^T X) theta = X^T y

def batch_gd(X, y, lr=0.1, epochs=1000):
    Xb = add_bias(X)
    theta = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        err = Xb @ theta - y
        theta -= lr * (2 / len(y)) * Xb.T @ err         # gradient step on MSE
    return theta

x = 2 * rng.random((100, 1)); y = 4 + 3 * x[:, 0] + rng.normal(0, 1, 100)    # truth: b=4, w=3
# ... then the 10-feature diabetes dataset, standardized, compared with sklearn
```

Output:

```
normal equation  [b, w]: [3.949 3.038]
batch GD         [b, w]: [3.949 3.038] (MSE 51.39 -> 0.959)
sklearn          [b, w]: [3.949 3.038]

method            test MSE   max |diff vs sklearn|      time
normal eq.          3424.3               1.46e-12    0.07ms
batch GD            3424.6               2.68e-01   35.06ms
sklearn             3424.3               0.00e+00
baseline (predict train mean) test MSE: 5134.8
GD after 5000 epochs, coef for s1, s2 (correlated pair): [-31.   15.1] exact: [-31.2  15.3]

largest standardized effects (+1 std in feature -> change in disease score):
    s5:   +35.8
    s1:   -31.2
   bmi:   +27.5
    s2:   +15.3
intercept = mean training target: 151.6 = 151.6
```

All three agree on the toy problem (3.949, 3.038 vs true 4 and 3; the gap is noise). On diabetes the normal equation matches scikit-learn to 12 decimals; GD is still slightly off after 5,000 epochs on `s1` / `s2`, two strongly correlated blood-serum measurements that make a long narrow valley in the **loss surface** where GD crawls (5.4). With standardized (mean 0) features the intercept equals the mean training target.

Trade-offs

	Normal equation / least squares	Gradient descent
Result	Exact, one shot	Approximate, iterative
Hyperparameters	None	Learning rate, epochs (batch size)
Cost	~$O(m \cdot d^2)$ to form X^TX + $O(d^3)$ to solve — fine for d up to a few thousand	$O(m \cdot d)$ per epoch; scales to huge m and d
Memory	Needs X^TX (d × d)	Can stream mini-batches from disk
Feature scaling	Not needed	Needed for fast convergence
Other losses	Only squared error (and Ridge)	Any **differentiable** (smooth, has a slope) loss — logistic regression, neural nets

(Big-O: how cost grows with m rows and d features; $O(d^3)$ means doubling features ≈ 8× slower solve.)

When not to use each

- Normal equation: not for very large d (tens of thousands) or data that doesn't fit in memory.
- Hand-rolled GD: not when a **closed form** (direct formula) or tuned library solver exists — use it for learning and for losses without closed forms.

Common mistakes

- `np.linalg.inv(X.T @ X)` explicitly — slower and numerically less stable (more rounding error) than `solve` / `lstsq`.
- Forgetting the column of ones; running GD on unscaled features and concluding "GD doesn't work".

Real-world examples

- scikit-learn `LinearRegression` (SciPy least squares); `statsmodels` **OLS** (ordinary least squares) for statistical inference (p-values, confidence intervals).
- `SGDRegressor` / `SGDClassifier` and Vowpal Wabbit fit linear models batch by batch with SGD, so data never needs to fit in RAM.
- Every neural network trains with GD variants (SGD with momentum, Adam: smoother step directions, per-weight step sizes).

In the interview

30 seconds: *"Linear regression minimises MSE. You can solve it exactly with the normal equation, $\theta = (X^TX)^{-1}X^Ty$ — in practice with a least-squares solver — which is cheap when there are few features. Gradient descent iterates $\theta \leftarrow \theta - \eta \cdot (2/m)X^T(X\theta - y)$; it scales to huge data, works for any differentiable loss, but needs a learning rate and scaled features."*

- *"When would the normal equation fail?"* — X^TX singular (duplicate/perfectly collinear features, or more features than rows); use the **pseudo-inverse** (generalised inverse via SVD, still least-squares) or Ridge, which makes $X^TX + \alpha I$ invertible (I = identity matrix).
- *"Is MSE convex?"* — Yes, for linear regression; GD with a small enough η reaches the global minimum.

Self-check

1. GD update for linear regression? → $\theta \leftarrow \theta - \eta\ (2/m)\ X^T(X\theta - y)$.
2. Why did GD lag on `s1` / `s2`? → Correlated features make a narrow valley.

TL;DR

Normal equation: exact, no tuning, $O(d^3)$ — great for small d. Gradient descent: iterative, scalable, general — needs a learning rate and scaled features.

5.4 Gradient descent in practice

What it is

Three flavours differing only in **how many examples compute each gradient**:

Variant	Examples per update	Updates per epoch (m = 100)	Behaviour
Batch GD	All m	1	Smooth, exact; slow per step on big data
Mini-batch GD	e.g. 16–512	m / batch size (100/16 → 7)	Noisy but efficient on vectorised hardware (GPUs); deep-learning default
Stochastic GD (SGD)	1	m (100)	Very noisy, cheap; can escape shallow local minima; never settles unless η decays

How it works

Shuffle each epoch; step on each batch's gradient; optionally decay η with a **learning-rate schedule**, e.g. $\eta_t = \eta_0 / (1 + k \cdot t)$, so noisy steps shrink.

```python
# code/gd_variants.py (key parts)
def run(Xb, y, lr, epochs, batch_size, seed=0, lr_decay=False):
    rng = np.random.default_rng(seed)
    theta, step = np.array([-1.0, -1.0]), 0
    for epoch in range(epochs):
        idx = rng.permutation(len(y))                    # reshuffle every epoch
        for start in range(0, len(y), batch_size):
            b = idx[start:start + batch_size]
            eta = lr / (1 + 0.05 * step) if lr_decay else lr
            theta = theta - eta * mse_grad(Xb[b], y[b], theta)
            step += 1
    return theta

run(Xb, y, lr=0.1, epochs=50, batch_size=100)            # batch
run(Xb, y, lr=0.1, epochs=8,  batch_size=16)             # mini-batch
run(Xb, y, lr=0.1, epochs=1,  batch_size=1, lr_decay=True)  # stochastic
```

Output:

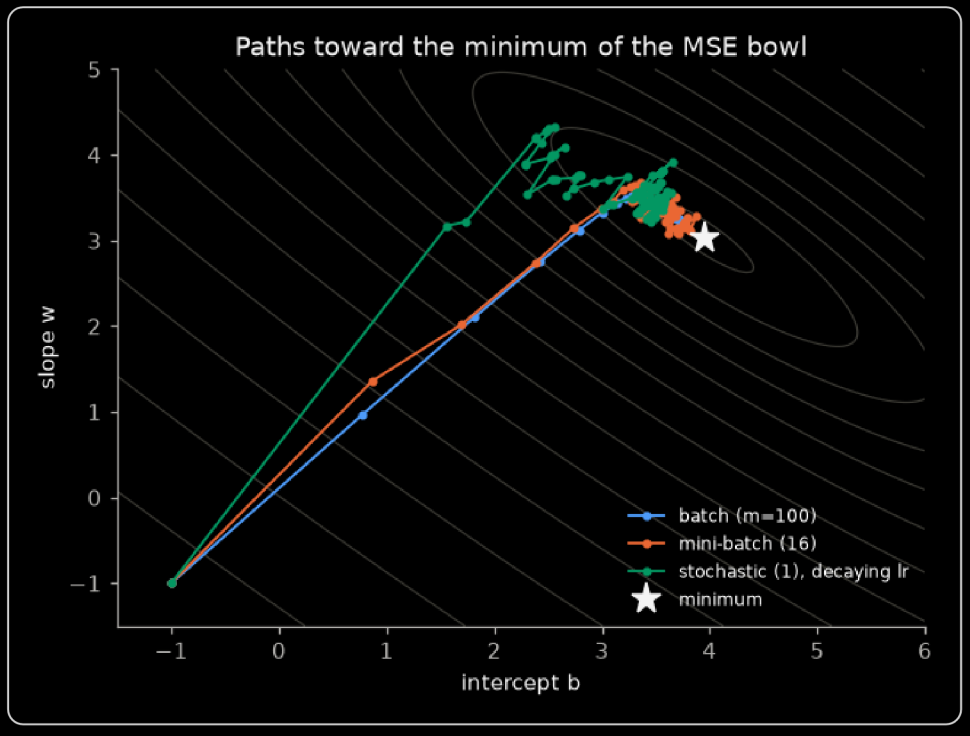
```
lr=0.01  MSE after 60 epochs: 1.37
lr=0.1   MSE after 60 epochs: 0.963
lr=0.6   MSE after 60 epochs: 1.92e+23

exact (normal eq.): [3.949 3.038]
batch (m=100)               updates=  50  final=[3.762 3.203]
mini-batch (16)             updates=  56  final=[3.814 3.109]
stochastic (1), decaying lr  updates= 100  final=[3.574 3.434]
unscaled      condition number        36  lr=1.36e-02  iterations to converge: 485
standardized  condition number         1  lr=4.86e-01  iterations to converge: 5
```

The learning rate

η = 0.01 crawls (MSE 1.37 after 60 epochs); 0.1 reaches the noise floor (≈ 0.96); 0.6 overshoots further every step and explodes to 10^{23} (**divergence**). For MSE, GD converges if $\eta < 2/\lambda_{max}$, where λ_{max} is the largest eigenvalue of the **Hessian** (curvature matrix, here $(2/m)X^TX$). In practice: try a log scale (0.001, 0.01, 0.1, …) and watch the loss curve.

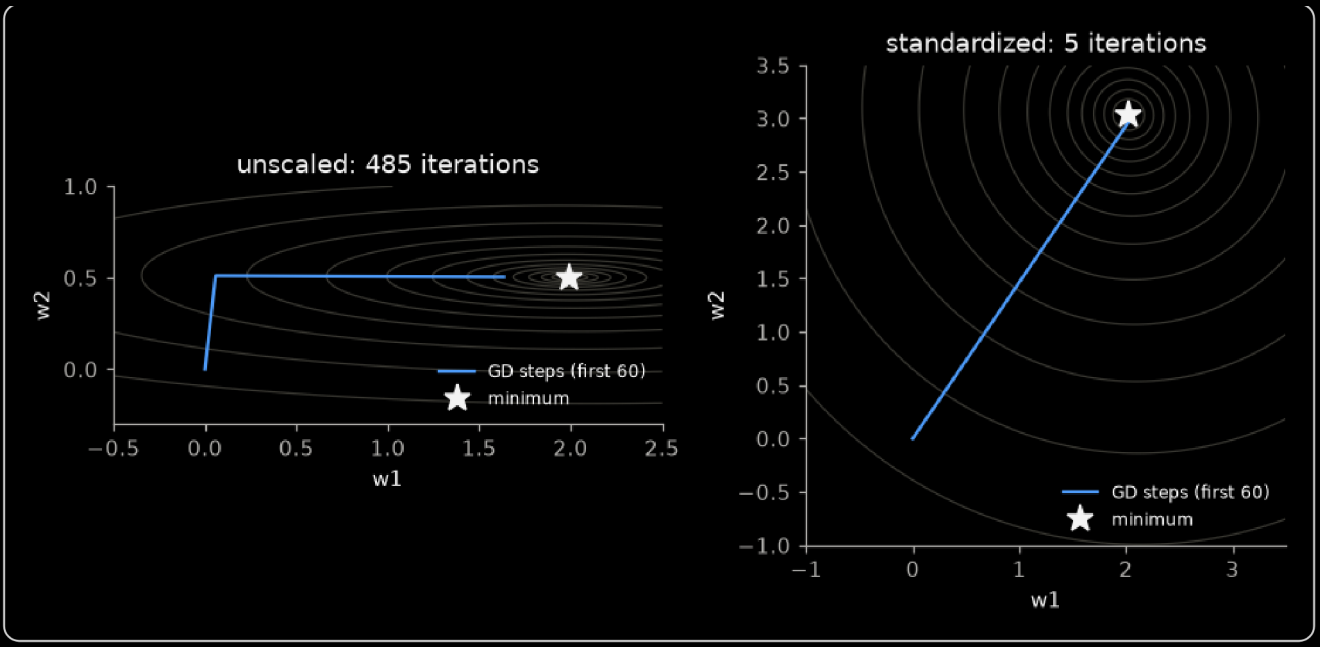




Batch moves smoothly, mini-batch wiggles, SGD bounces and settles only because η decays. With equal update counts batch GD saw 50× more data — per unit of *compute*, SGD and mini-batch are far cheaper on large data, which is why they dominate.

Why feature scaling matters for GD

Unequal spreads turn the MSE bowl into a narrow valley: η must be small for the steep direction, so progress along the flat one crawls. The **condition number** (λ_max / λ_min) measures the stretch; GD's step count grows roughly with it.



A 6× spread gives condition number 36 and **485** iterations; standardized, the bowl is round (≈ 1) and GD needs **5**. At 100× spread, well over 100,000. Correlated features (`s1` / `s2`) stretch the bowl too; scaling doesn't fully fix that — regularization helps.

Trade-offs / common mistakes

- Larger batches: smoother gradients, better hardware use, fewer updates per epoch; very large ones may need a larger η or **warm-up** (ramp η up over the first steps).
- Not shuffling → unrepresentative batches (e.g. sorted by class).
- Judging SGD by its last iterate without η decay — it keeps bouncing.
- Fixed epoch counts instead of monitoring validation loss (**early stopping**, Chapter 6).

In the interview

30 seconds: *"Batch GD uses all data per step — smooth but expensive; SGD uses one example — cheap and noisy, needs a decaying learning rate; mini-batch is the practical middle ground and fits GPUs. The learning rate is the key hyperparameter: too small is slow, too large diverges. Scale features first — otherwise the loss surface is a narrow valley and GD zig-zags or crawls."*

- *"Why does SGD sometimes generalise better?"* — Its noise acts like a **regularizer** (anything discouraging overfitting; 5.7) and can escape sharp minima — an empirical observation, not a guarantee.
- *"How do you pick a learning rate?"* — Log-scale search on the training-loss curve, or a learning-rate range test (raise η each step, note where loss blows up); then add a schedule.

Self-check

1. m = 1,000, batch 50: updates per epoch? → 20.

TL;DR

Batch = exact, slow; SGD = noisy, cheap; mini-batch = default. Tune η on a log scale, decay it for SGD, scale features.

5.5 Assumptions of linear regression

What it is

Linear regression always returns *some* line; whether predictions are good and **coefficients and p-values** trustworthy depends on assumptions:

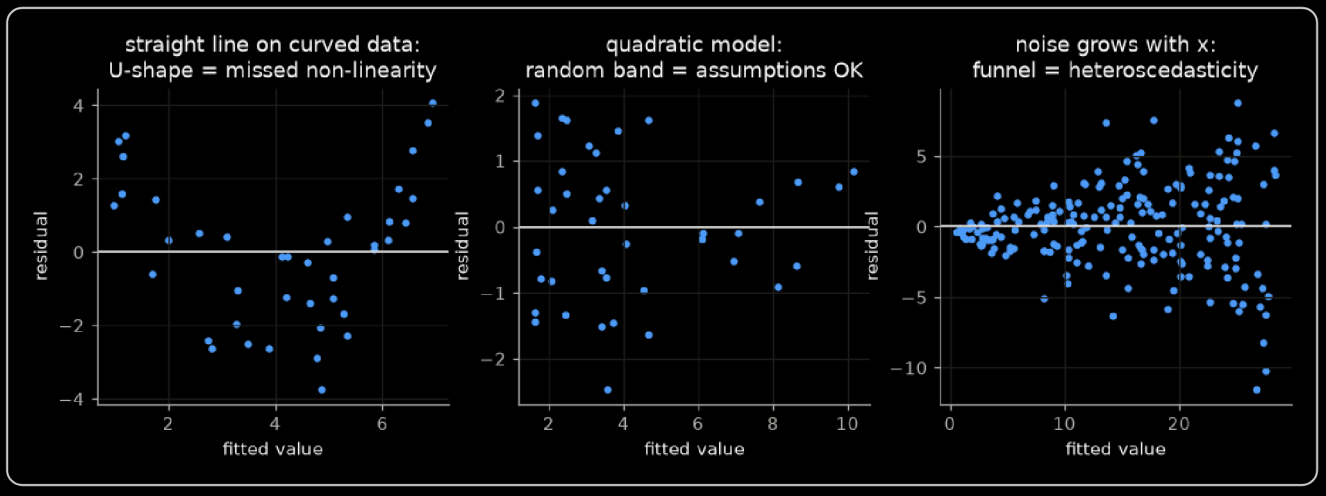
Assumption	Meaning	If violated
Linearity	y is linear in the (possibly transformed) features	Systematic errors; add polynomial/log terms or go non-linear
Independence	Row errors unrelated	Common in time series or repeated measures (several rows per subject); **standard errors** (coefficient uncertainty) too small
Homoscedasticity	Constant error spread	"Funnel" residuals; coefficients still unbiased but standard errors wrong; try log(y) or weighted least squares (noisier rows count less)
Normal errors	Residuals roughly normal	Matters mainly for small-sample CIs and p-values
No (perfect) multicollinearity	Features aren't near-copies	Unstable, uninterpretable coefficients (predictions can be fine)

For prediction, linearity and a stable data distribution matter most; the rest matter when interpreting coefficients or reporting **statistical significance**.

How to check: residual plots

A **residual** is $y_i - \hat{y}_i$. Against fitted values, a good model leaves a structureless band around 0.

```
# code/polynomial_and_assumptions.py (residual part)
lin = LinearRegression().fit(x[:, None], y)           # straight line on quadratic data
resid = y - lin.predict(x[:, None])
ax.scatter(lin.predict(x[:, None]), resid); ax.axhline(0)
```



- **U-shape:** missed curvature — add x^2 or transform.
- **Random band:** the quadratic model captures the pattern.
- **Funnel:** noise grows with x (heteroscedasticity) — log y, weighted least squares, or **robust standard errors** (valid under varying spread).

Common mistakes

- Checking only **R^2** (share of target variance explained) and never plotting residuals.
- Reading coefficients causally from **observational data** (recorded as it happened, not from an experiment).

In the interview

30 seconds: *"Linear regression assumes a linear relationship, independent errors with constant variance, roughly normal errors, and no strong multicollinearity. For prediction, linearity and a stable distribution matter most; the rest matter when you interpret coefficients or p-values. I check with a residuals-vs-fitted plot."*

- *"Funnel-shaped residuals?"* — Heteroscedasticity; log y, weighted least squares or robust standard errors.

Self-check

1. Does heteroscedasticity bias coefficients? → No; it makes their standard errors wrong.
2. Does multicollinearity hurt predictions? → Usually not much; it destabilises individual coefficients.

TL;DR

Plot residuals vs fitted: a U means missing non-linearity, a funnel non-constant variance; correlated features make coefficients unreliable.

5.6 Polynomial regression

What it is

Linear regression on polynomial features (degree 2, one feature: $\hat{y} = w_2x^2 + w_1x + b$). Still "linear" — in the *weights* — so the same solvers work; only inputs are expanded (`PolynomialFeatures`).

Why it matters

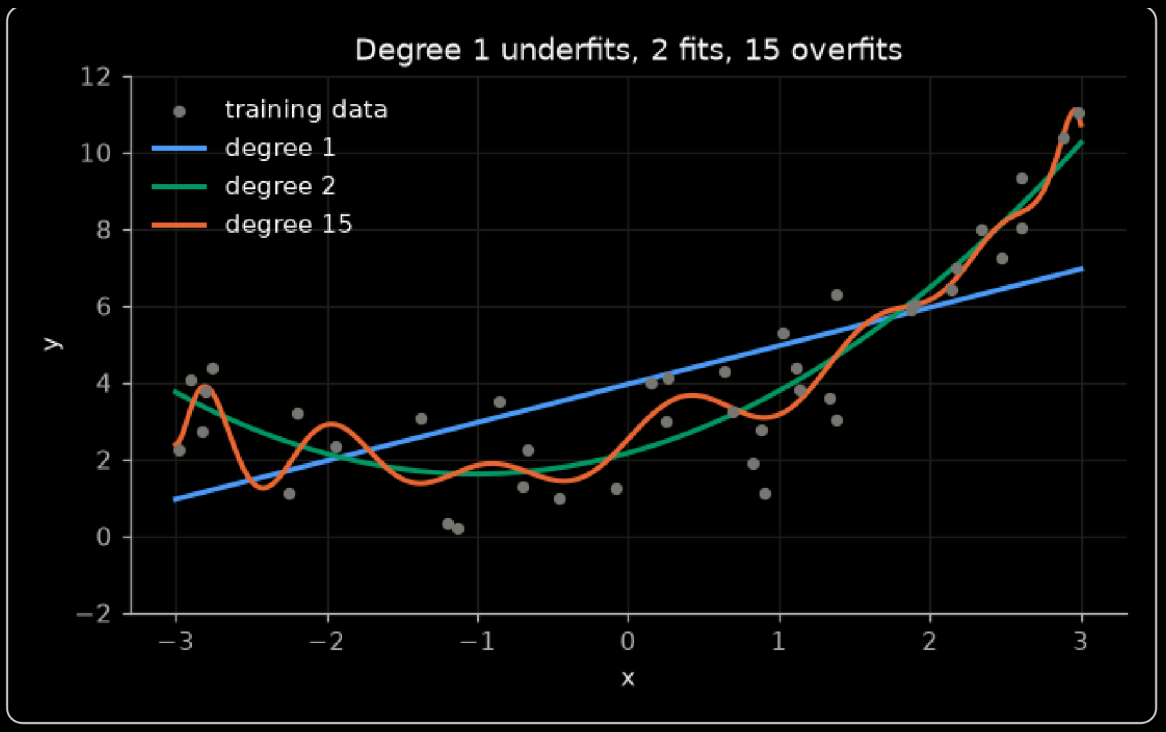
The simplest way to fit curves and the clearest demonstration of:

- **Underfitting** — too simple: high error on training *and* test (high **bias**: systematic error from a too-rigid model).
- **Overfitting** — fits the noise: low training, higher test error (high **variance**: fit changes a lot with the training sample).

```
# code/polynomial_and_assumptions.py (key parts) — truth: y = 0.5x² + x + 2 + noise, 40 training points
for deg in [1, 2, 15]:
    model = make_pipeline(PolynomialFeatures(deg, include_bias=False), StandardScaler(), LinearRegression())
    model.fit(x[:, None], y)
```

Output:

```
degree  train MSE  test MSE
     1       3.73      2.83
     2       1.14      1.04
    15       0.90      1.35
degree-2 learned: y = 0.54 x^2 + 1.08 x + 2.16   (truth 0.5, 1, 2)
```



Degree 2 recovers the true coefficients. Degree 15 has the *lowest training error* but higher test error, wiggling between points and shooting off at the edges (test points here lie inside the training range; outside it, degree 15 is wildly wrong). Noise variance is 1.0, so test MSE ≈ 1.0 is the best possible.

Trade-offs / when not to use it

- Features explode: d features at degree k → C(d+k, k) − 1 columns (10 at degree 2 → 65; degree 3 → 285).
- High degrees are unstable at the edges and extrapolate badly; prefer low degrees, **splines** (smooth curves of short low-degree pieces joined end to end; `SplineTransformer`) or trees/boosting.
- Scale after expanding: x^{15} dwarfs x.

In the interview

30 seconds: *"Polynomial regression expands the inputs with powers and products and fits an ordinary linear model — linear in the weights. The degree is a complexity knob: too low underfits, too high overfits and extrapolates wildly. I'd choose the degree, or add Ridge, using validation data."*

Self-check

1. Degree-2 features from 3 inputs (no bias)? → 3 + 3 squares + 3 pairwise = 9.

TL;DR

Expand features with powers and products, fit a linear model; pick the degree (or regularize) on validation data.

5.7 Regularization: Ridge, Lasso, ElasticNet

What it is

Regularization adds a weight-size penalty to the loss, so the model only "spends" weight where data strongly justifies it.

Model	Penalty	scikit-learn objective	Effect
Ridge (L2)	$\alpha \sum w_j^2$	$\|y - Xw\|^2 + \alpha\|w\|_2^2$	Shrinks all weights smoothly toward 0, never exactly 0; splits weight among correlated features
Lasso (L1)	$\alpha \sum \|w_j\|$	$(1/2m)\|y - Xw\|^2 + \alpha\|w\|_1$	Sets some weights **exactly 0** → automatic feature selection; picks one of a correlated group
ElasticNet	Both	$(1/2m)\|y - Xw\|^2 + \alpha \cdot r\|w\|_1 + \frac{1}{2}\alpha(1 - r)\|w\|_2^2$	**Sparse** (many exact zeros) *and* stable with correlated features; r = `l1_ratio`

- **α** = strength: 0 = plain least squares; large → weights toward 0. Ridge's and Lasso's α are on different scales in scikit-learn (sum vs divided by 2m), so the same α shrinks differently.
- The **intercept is not penalized**. **Scale features first** — the penalty treats all weights equally.
- Logistic regression and SVMs (Chapter 8) use **C = 1/strength**: *smaller* C = *stronger* regularization.

Why it matters

- Reduces overfitting with many features relative to rows, or correlated features.
- Makes $X^TX + \alpha I$ always invertible, so Ridge has a closed form $\theta = (X^TX + \alpha I)^{-1}X^Ty$ (intercept unpenalized).
- Lasso gives compact, interpretable models.

Why L1 gives exact zeros and L2 doesn't: geometrically, the L1 "budget" is a diamond with corners on the axes, and loss contours usually first touch it at a corner (some weights 0); the L2 circle has no corners. Computationally, L1's slope is a constant ±α however small the weight, pushing it all the way to 0; L2's slope 2αw fades as w shrinks.

```
# code/regularization.py (key parts) — diabetes data, standardized
ridge_path = np.array([Ridge(alpha=a).fit(X, y).coef_ for a in np.logspace(-2, 4, 60)])
lasso_path = np.array([Lasso(alpha=a, max_iter=50_000).fit(X, y).coef_ for a in np.logspace(-2, 2, 60)])

X_dup = np.c_[X[:, [2]], X[:, [2]] + rng.normal(0, 0.01, (len(X), 1))]    # bmi twice (near-copy)

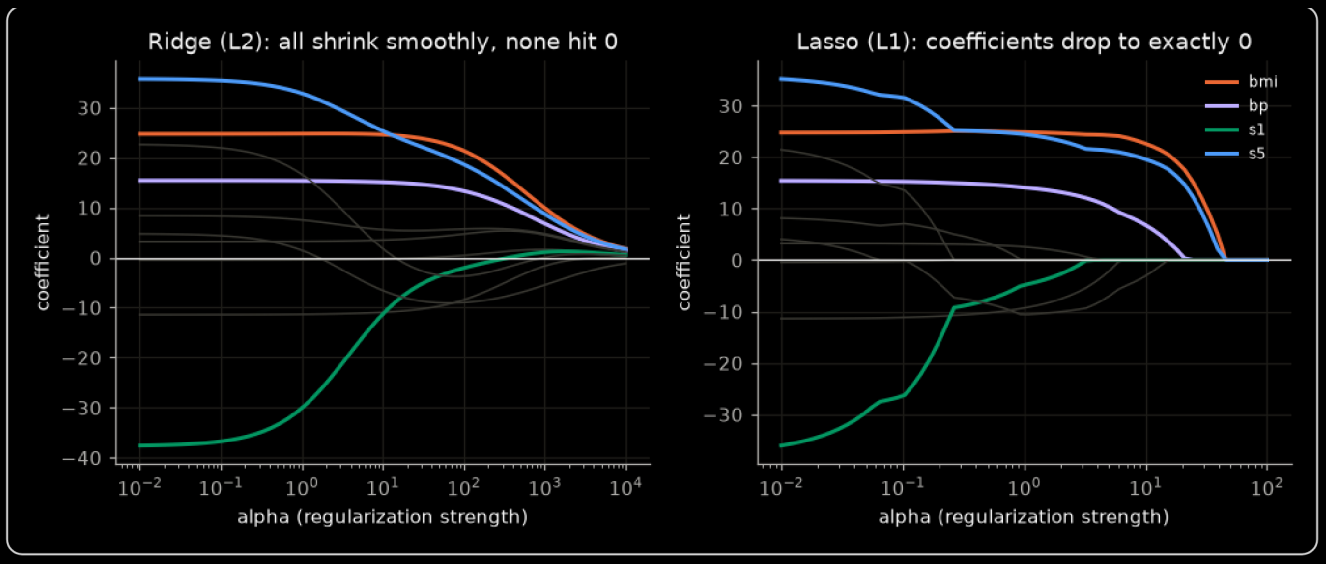
poly = PolynomialFeatures(2, include_bias=False)                          # 65 features, ~330 rows
models = {"OLS": make_pipeline(poly, StandardScaler(), LinearRegression()),
          "RidgeCV": make_pipeline(poly, StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 50))),
          "LassoCV": make_pipeline(poly, StandardScaler(), LassoCV(cv=5, random_state=0)),
          "ElasticNetCV": make_pipeline(poly, StandardScaler(),
                                        ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=5, random_state=0))}
```

Output:

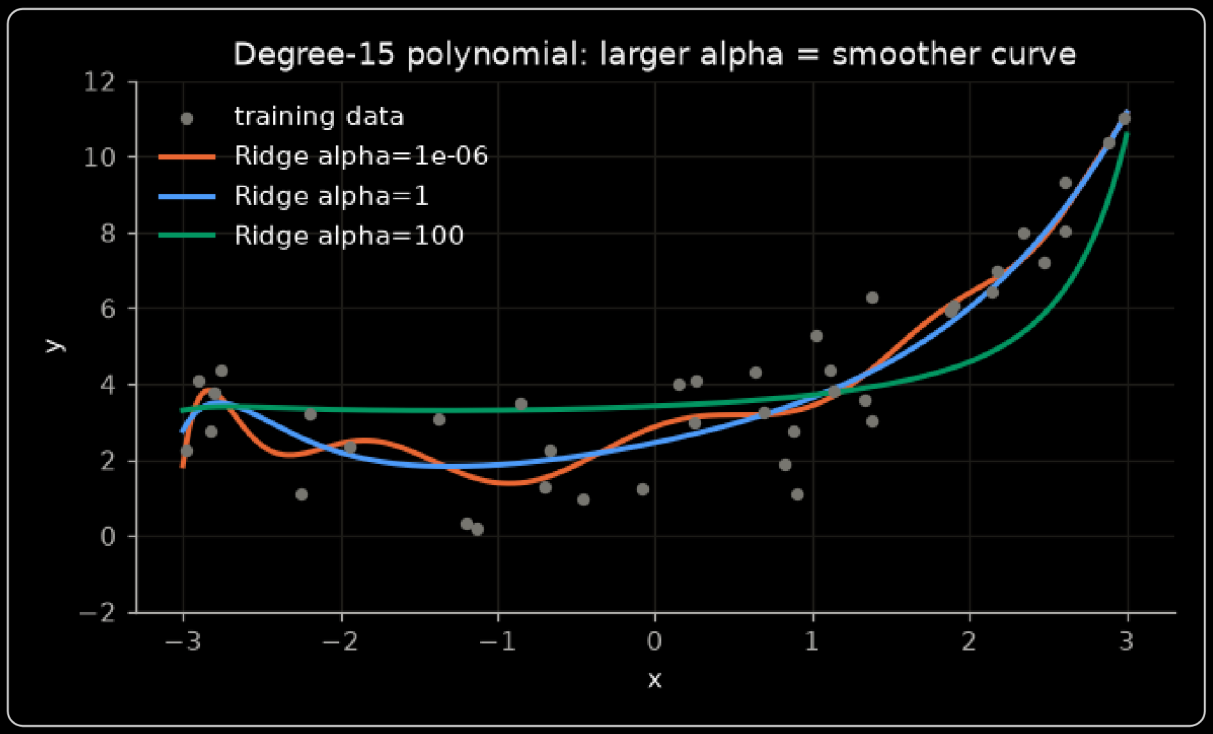
```
Lasso alpha=0.1 :  9 of 10 coefficients non-zero
Lasso alpha=1   :  7 of 10 coefficients non-zero
Lasso alpha=5   :  5 of 10 coefficients non-zero
Lasso alpha=20  :  3 of 10 coefficients non-zero

bmi duplicated -> OLS: [ 434.3 -389.1]  Ridge(1): [31.5 13.5]  Lasso(1): [44.2  0. ]

model          test MSE    alpha l1_ratio non-zero
OLS                4112      nan      nan       65
RidgeCV            3291   59.636      nan       65
LassoCV            3113    4.256      nan       11
ElasticNetCV       3178    0.754      0.8       41
```



- **Sparsity:** as Lasso's α grows, 9 → 7 → 5 → 3 non-zero; the last survivors (`bmi`, `s5`) are the strongest predictors.
- **Correlated features:** with `bmi` duplicated, OLS gives absurd opposite-signed weights (+434, −389) that nearly cancel; Ridge shares (31.5 + 13.5 ≈ 45); Lasso keeps one (44.2).
- **Generalisation:** 65 polynomial features on ~330 rows: OLS overfits (4112); CV-tuned Ridge/Lasso/ElasticNet ≈ 3100–3300, Lasso keeping 11 of 65. (Honest footnote: OLS on the *original* 10 features scores ~2900 — regularization rescues an over-expanded model but doesn't make needless features useful.)



Trade-offs

	Ridge	Lasso	ElasticNet
Feature selection	No	Yes	Yes (less aggressive)
Correlated features	Shares weight — stable	Picks one arbitrarily — unstable	Groups them — stable
Closed form	Yes	No (coordinate descent: one weight at a time)	No
When p > n (more features than rows)	Works	Selects at most n features	Works
Good default when	Many small effects	Few features truly matter	Correlated groups + want sparsity

When not to use it

With plenty of rows per feature it matters little (diabetes' raw 10 features: all four within ~2%, test MSE ≈ 2900–2950). Don't treat Lasso's selected set as "the true causes" — with correlated features its choice flips between resamples.

Common mistakes

- Regularizing unscaled features; choosing α on the test set (use `RidgeCV` / `LassoCV` / `GridSearchCV`); forgetting C in `LogisticRegression` is the *inverse* strength.

Real-world examples

- Ridge is the classic multicollinearity fix in econometrics; L2 "weight decay" is standard in neural-network training.
- L1-regularized logistic regression for sparse, high-dimensional text and click models (Google's 2013 FTRL-Proximal ad click model, an online method updating per example, used L1 for sparsity).
- Genomics (thousands of genes, few patients) uses Lasso/ElasticNet to select predictive genes.

In the interview

30 seconds: *"Regularization adds a weight penalty to fight overfitting. Ridge (L2) shrinks all weights smoothly and handles correlated features; Lasso (L1) zeroes some weights, doing feature selection; ElasticNet mixes both. Scale features first and tune α with cross-validation."*

- *"Why does Lasso give zeros?"* — Constant L1 slope pushes small weights to 0; the diamond constraint has corners on the axes.
- *"Two identical features?"* — Ridge splits the weight equally; Lasso picks one.
- *"Effect on bias and variance?"* — More regularization → more bias, less variance.

Self-check

1. `LogisticRegression` C = 0.01: strong or weak? → Strong.

TL;DR

Penalize big weights: L2 shrinks, L1 selects, ElasticNet both. Scale first, tune α (or C) by cross-validation.

5.8 Logistic regression

What it is

Logistic regression is a linear model for **binary classification**. It computes z = w·x + b (the **logit**) and squashes it with the **sigmoid** (logistic) function:

p = P(y = 1 | x) = σ(z) = 1 / (1 + e^−z)

Despite the name, it's a classifier (named after the logistic function).

How it works

1. Sigmoid maps reals to (0, 1): σ(−2) = 0.119, σ(0) = 0.5, σ(2) = 0.881.

2. Log-odds. Inverting: z = ln(p / (1 − p)); p/(1 − p) is the **odds** (p = 0.8 → 4 to 1). The model is linear in **log-odds**: +1 in x_j adds w_j to the log-odds, **multiplying the odds by e^{w_j}** (the **odds ratio**).

3. Decision boundary. Predict 1 when p ≥ threshold (default 0.5). p = 0.5 exactly when z = 0, so the **decision boundary** is w·x + b = 0 — a line (2-D) or hyperplane. Changing the threshold shifts it parallel (choosing thresholds: Chapter 6).

4. Log-loss (binary cross-entropy). Squared error with a sigmoid is non-convex (several dips to get stuck in), so:

Loss = −(1/m) Σ [y_i ln p_i + (1 − y_i) ln(1 − p_i)]

Only one term is active per example: −ln p if y = 1, −ln(1 − p) if y = 0.

True y	Predicted p	Loss	Reading
1	0.9	−ln 0.9 = 0.105	confident and right: tiny loss
1	0.5	−ln 0.5 = 0.693	unsure
1	0.1	−ln 0.1 = 2.303	confident and wrong: big loss
0	0.1	−ln 0.9 = 0.105	confident and right

It is the negative log-likelihood under a **Bernoulli** model (one yes/no trial, 1 with probability p), so minimising it is maximum likelihood. Convex in the weights (GD finds the global minimum) but no closed form — solved iteratively.

5. Gradient — same form as linear regression's: ∇Loss = (1/m) X^T(σ(Xθ) − y), "features × (prediction − truth)". For one example (p = σ(z), z = θ^Tx): dLoss/dp = −y/p + (1 − y)/(1 − p); dp/dz = p(1 − p); so dLoss/dz = **p − y**, and dz/dθ_j = x_j. (No factor 2: no square.)

```python
# code/logreg_from_scratch.py (key parts)
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def fit_logreg_gd(X, y, lr=0.5, epochs=5000):
    Xb = np.c_[np.ones(len(X)), X]
    theta = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        p = sigmoid(Xb @ theta)
        theta -= lr * Xb.T @ (p - y) / len(y)            # gradient of mean log-loss
    return theta

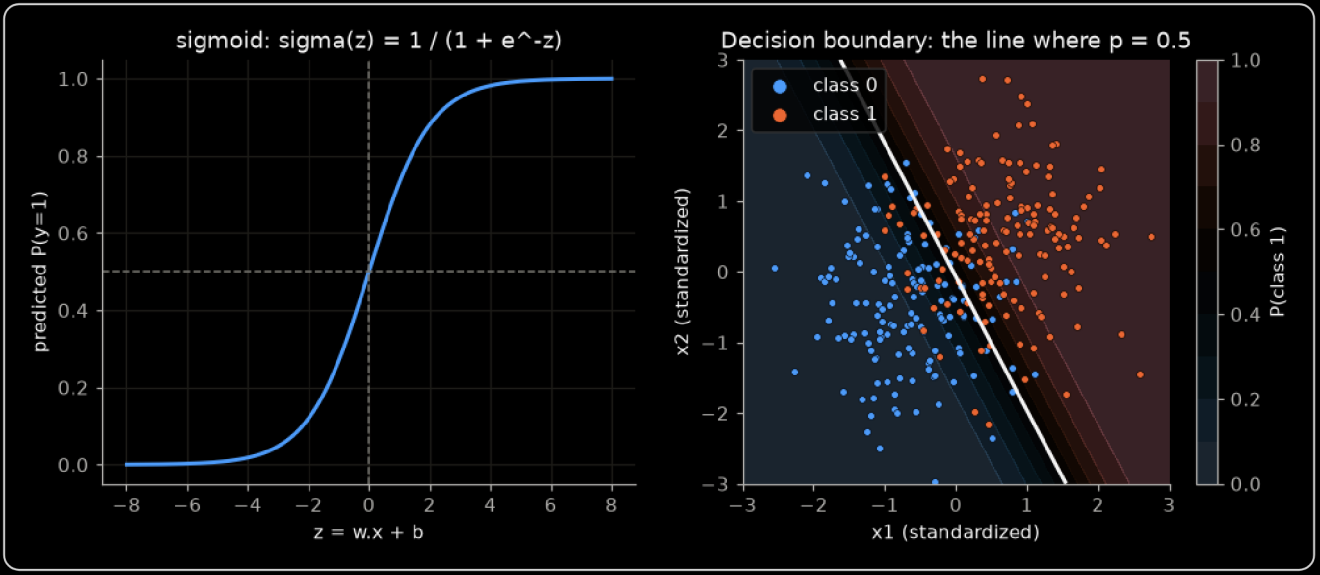
X, y = make_blobs(n_samples=400, centers=[[-1, -0.5], [1, 0.8]], cluster_std=1.1, random_state=0)
# ... stratified split, StandardScaler fitted on train
theta = fit_logreg_gd(X_tr, y_tr)
sk = LogisticRegression(C=np.inf).fit(X_tr, y_tr)        # C=inf -> no penalty (sklearn >= 1.8)
```

Output:

```
sigmoid(-2) = 0.119
sigmoid(+0) = 0.500
sigmoid(+2) = 0.881
y=1, p=0.9: loss = 0.105
y=1, p=0.5: loss = 0.693
y=1, p=0.1: loss = 2.303
y=0, p=0.1: loss = 0.105

from scratch [b, w1, w2]: [0.092 2.454 1.297]
sklearn      [b, w1, w2]: [0.092 2.454 1.298]
test log-loss scratch 0.4048 | sklearn 0.4048
test accuracy scratch 0.850 | sklearn 0.850
w1 = 2.454: +1 std in x1 multiplies the odds of class 1 by e^w1 = 11.64
```

Ten lines of GD match scikit-learn's **L-BFGS** solver (a faster optimiser using a curvature estimate) to ~three decimals.



Probability changes smoothly across the boundary; points near it get ≈ 0.5 — honest uncertainty about overlapping classes.

scikit-learn details (1.8+). `LogisticRegression` is L2-regularized by default (`C=1.0`, solver `lbfgs`). The old `penalty=` argument is deprecated: `C=np.inf` = no penalty; `l1_ratio=1` (with `solver="saga"` or `"liblinear"`) = L1; values in (0, 1) = ElasticNet.

Trade-offs

Strengths	Weaknesses
Fast to train and serve, tiny model	Linear boundary only (needs feature engineering or kernels)
Probabilities often reasonably **calibrated** (predicted 0.7 comes true ~70% of the time)	Sensitive to correlated features and outliers in X
Coefficients readable as odds ratios	On **perfectly separable** data, unregularized weights grow without bound
Convex: one global optimum	Usually beaten by gradient boosting (Chapter 7) on complex tabular data

When not to use it

Strongly non-linear boundaries without good features (use trees/boosting or neural nets); high-cardinality interactions you can't enumerate.

Common mistakes

- `LinearRegression` on a 0/1 target — predictions leave [0, 1], fit dominated by easy points.

- Forgetting the default is regularized (C = 1): compare coefficients only with the same C and scaling.
- Assuming 0.5 is the right threshold on imbalanced data.

Real-world examples

- **Credit scoring** scorecards — regulators and loan officers must read the coefficients.
- **Ad click-through rate** at Google and Facebook (Facebook's 2014 paper fed boosted-tree features into logistic regression).
- Medical risk scores; the final layer of every neural binary classifier (sigmoid on a linear layer).

In the interview

30 seconds: *"Logistic regression computes a linear score $z = w \cdot x + b$, turns it into a probability with the sigmoid, and is trained by minimising log-loss — convex, so gradient descent finds the global optimum. The decision boundary is the hyperplane $w \cdot x + b = 0$. Each coefficient is a change in log-odds, so e^{w} is an odds ratio."*

- *"Why not MSE for classification?"* — With a sigmoid it's non-convex and gives weak gradients for confident mistakes; log-loss is convex and punishes them hard.
- *"Perfectly separable data?"* — Loss keeps falling as weights grow, so they diverge; regularization keeps them finite.
- *"Calibrated?"* — Often roughly, since log-loss is a **proper scoring rule** (lowest only at the true probabilities); check with a **reliability curve** (predicted probability vs observed frequency; Chapter 6).

Self-check

1. Loss for $y = 1$, $p = 0.25$? → 1.386.
2. $w = 0.7$: effect on odds? → × $e^{0.7} \approx 2.01$ per unit.

TL;DR

Linear score → sigmoid → probability; log-loss (convex, gradient $X^{T}(p - y)/m$); boundary $w \cdot x + b = 0$; coefficients are log-odds.

5.9 Multiclass classification: one-vs-rest and softmax

What it is

Two ways to handle $K > 2$ classes:

1. **One-vs-rest (OvR)** — K binary classifiers ("class k vs the rest"); predict the most confident.
2. **Softmax (multinomial) regression** — one model, a weight vector per class, scores $z_k = w_k \cdot x + b_k$ turned into probabilities summing to 1 by **softmax**: $P(y = k \mid x) = e^{z_k} / \Sigma_j e^{z_j}$.

Worked example: scores (2, 1, 0.1) → $e^{2} = 7.39$, $e^{1} = 2.72$, $e^{0.1} = 1.11$, total 11.21 → **(0.659, 0.242, 0.099)**.

The loss is **cross-entropy** $-(1/m) \Sigma_i \ln P(y_i \mid x_i)$ (negative log-probability of the correct class); with $K = 2$ it is exactly binary logistic regression. Gradient: $X^{T}(P - Y)/m$, Y = one-hot labels.

```python
# code/multiclass.py (key parts)
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)                # subtract row max: avoids overflow
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)

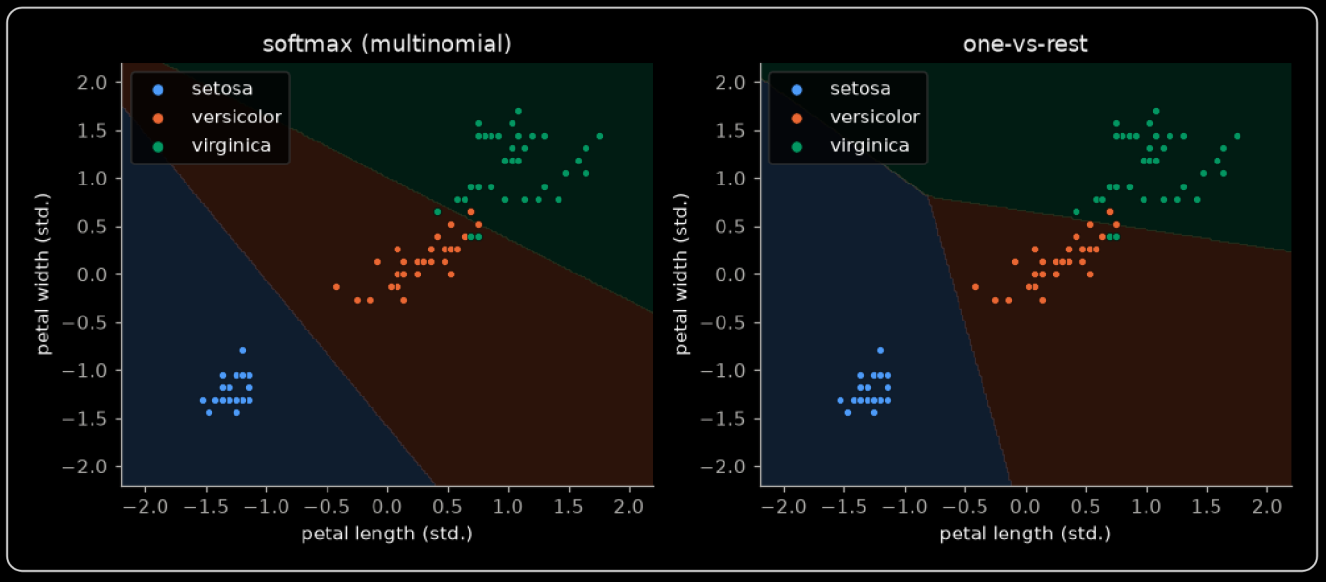
def fit_softmax_gd(X, y, k, lr=0.5, epochs=3000, l2=0.0):
    Xb = np.c_[np.ones(len(X)), X]
    W = np.zeros((Xb.shape[1], k)); Y = np.eye(k)[y]      # one-hot targets
    for _ in range(epochs):
        grad = Xb.T @ (softmax(Xb @ W) - Y) / len(y)
        grad[1:] += l2 * W[1:]
        W -= lr * grad
    return W

softmax_sk = LogisticRegression(C=10).fit(X_tr, y_tr)                  # multinomial by default
ovr = OneVsRestClassifier(LogisticRegression(C=10)).fit(X_tr, y_tr)     # 3 binary models
```

Output:

```
softmax([2, 1, 0.1]) = [[0.659 0.242 0.099]]  sum = 1.0

test accuracy  softmax(sklearn) 0.978 | OvR 0.978 | softmax(scratch) 0.978
one flower, P(setosa, versicolor, virginica):
  softmax sklearn : [[0.    0.001 0.999]]
  softmax scratch : [0.    0.001 0.999]
  OvR raw scores  : [0.    0.175 0.998] (sum 1.174) -> normalized [[0.    0.149 0.851]]
sklearn softmax coef_ shape: (3, 2) (one weight vector per class)
```



Same 97.8% accuracy, different probabilities: independently trained OvR scores sum to 1.174 and are simply rescaled (0.851 for virginica vs softmax's 0.999); its boundaries come from three separate problems.

Trade-offs

	One-vs-rest	Softmax
Models	K binary	One joint
Probabilities	Not naturally normalised	Sum to 1, jointly trained — better calibrated
Parallel training / per-class tuning	Easy	No
Use with	Any binary classifier (e.g. SVMs); multi-label	Mutually exclusive classes; neural output layers

One-vs-one trains K(K−1)/2 pairwise classifiers (used inside `SVC`). For **multi-label** problems (several labels per item, e.g. photo tags) use independent sigmoids, not softmax. scikit-learn 1.9 has no `multi_class` argument: `LogisticRegression` fits softmax for K > 2; wrap in `OneVsRestClassifier` for OvR.

In the interview

30 seconds: *"For K classes you can train K one-vs-rest binary models, or one softmax model that outputs K scores and normalises them with e^z / Σe^z, trained with cross-entropy. Softmax gives coherent probabilities for mutually exclusive classes and is the output layer of neural classifiers; OvR works with any binary model and suits multi-label problems."*

- *"Why subtract the max in softmax?"* — e^1000 overflows; subtracting a constant from all scores doesn't change the result.
- *"Softmax with 2 classes?"* — Logistic regression on the score difference.

Self-check

1. softmax(0, 0, 0)? → (1/3, 1/3, 1/3).
2. Classifiers for 10 classes? → OvR 10; one-vs-one 45.

TL;DR

OvR = K binary models; softmax = one model, normalised probabilities, cross-entropy. Softmax for exclusive classes, sigmoids for multi-label.

5.10 Interpreting coefficients

What it is

Coefficients are a linear model's explanation; reading them depends on units, scaling and correlations.

How it works

- **Raw units:** w_j = change in ŷ per unit of x_j, **others fixed**. Switch m² → km² and the number changes a millionfold; the model doesn't.
- **Standardized features:** w_j = change per 1 standard deviation — comparable across features, a rough importance ranking.
- **Logistic:** w_j = change in log-odds; e^w_j = **odds ratio** (> 1 raises odds, < 1 lowers).
- **Intercept:** prediction at all-zero features (standardized: at average values).

```
# code/interpret_coefficients.py (key parts)
price = 50 + 3.0 * size - 1.5 * age + rng.normal(0, 20, n)     # $1000s, size in m², age in years
LinearRegression().fit(np.c_[size, age], price)
LinearRegression().fit(np.c_[size / 1e6, age], price)           # size in km²
make_pipeline(StandardScaler(), LinearRegression()).fit(np.c_[size, age], price)

clf = make_pipeline(StandardScaler(), LogisticRegression(C=0.1, max_iter=5000)).fit(cancer.data, cancer.target)

x2 = x1 + rng.normal(0, 0.05, 300)                               # near-duplicate feature
# fit on 3 bootstrap resamples
```

Output:

```
raw units      : price = 52.6 + 2.98*size_m2 -1.49*age_years
size in km^2   : coef for size = 2984148  (same model, different units -> different number)
standardized   : +1 std of size -> +137.6, +1 std of age -> -20.8  (comparable importance)

breast cancer (target 1 = benign), standardized features, C=0.1:
  worst texture            coef -0.60  odds ratio e^coef = 0.55
  worst radius             coef -0.54  odds ratio e^coef = 0.58
  worst concave points     coef -0.53  odds ratio e^coef = 0.59
  fractal dimension error  coef +0.27  odds ratio e^coef = 1.31
  compactness error        coef +0.27  odds ratio e^coef = 1.31

bootstrap coefficients for two near-duplicate features (true: 1.0 and 1.0):
  resample 0: w1 = +1.07, w2 = +0.95, sum = 2.02
  resample 1: w1 = +2.91, w2 = -0.92, sum = 2.00
  resample 2: w1 = -1.03, w2 = +3.04, sum = 2.01
```

- Each m² adds ~$2,980, each year of age subtracts ~$1,490. Raw, age looks half as big as size; standardized, size matters ~6.6× more (137.6 vs 20.8) because sizes vary much more.
- One std more "worst texture" multiplies the odds of *benign* by 0.55 (about halves them).
- Near-duplicate features: across **bootstrap** resamples (rows redrawn at random with replacement) individual coefficients swing wildly, even flipping sign, while their **sum** stays ≈ 2. Predictions are fine; individual numbers are meaningless. Diagnose with correlations or the **variance inflation factor (VIF)** (how much a coefficient's uncertainty is inflated by correlation with other features); fix by dropping/combining features or Ridge.

Common mistakes

- Comparing raw coefficients across different units; reading them causally from observational data.
- Interpreting one coefficient of a correlated group alone; forgetting regularization biases coefficients toward 0.

In the interview

30 seconds: *"A coefficient is the change in prediction per unit of that feature, holding the others constant. Standardize features to compare magnitudes; in logistic regression exponentiate (compute e to the power of the coefficient) to get odds ratios. Coefficients are unstable under multicollinearity and are associations, not causal effects."*

Self-check

1. Logistic coefficient −0.69? → × e^−0.69 ≈ 0.5: halves the odds.
2. Why are correlated coefficients unstable? → Many weight combinations predict almost identically; small data changes flip between them.

TL;DR

Coefficients = effect per unit (or per std if standardized), others fixed; exponentiate logistic ones for odds ratios; distrust individual coefficients of correlated features and never read them causally.

Chapter TL;DR

- **Workflow:** data → clean → encode (one-hot for nominal) → split → fit → evaluate against a baseline → iterate. MSE only means something relative to the target's scale and a baseline.
- **Linear regression:** $\hat{y} = w \cdot x + b$, minimise MSE (convex). **Normal equation** $\theta = (X^T X)^{-1} X^T y$ via a solver — exact, $O(d^3)$, no scaling. **Gradient descent** $\theta \leftarrow \theta - \eta(2/m)X^T(X\theta - y)$ — scalable, general, needs η and scaled features.
- **GD variants:** batch (smooth, costly), stochastic (noisy, decaying η), mini-batch (default). Too-large η diverges (MSE: $\eta < 2/\lambda_{max}$). Unscaled features → elongated bowl → many more iterations.
- **Assumptions:** linearity, independent errors, constant variance, roughly normal errors, no strong multicollinearity; check residual plots (U = missing curvature, funnel = heteroscedasticity).

- **Polynomial regression:** linear model on expanded features; degree controls under/overfitting.
- **Regularization:** Ridge (L2, shrink, stable with correlated features), Lasso (L1, exact zeros), ElasticNet (both). Scale first; tune α (or C = 1/strength) by CV.
- **Logistic regression:** $p = \sigma(w \cdot x + b)$, log-loss (convex, gradient $X^T(p - y)/m$), boundary $w \cdot x + b = 0$, e^w = odds ratio. sklearn ≥ 1.8: `C=np.inf` = no penalty.
- **Multiclass:** OvR (K binary models) vs softmax ($e^{z_k}/\Sigma e^{z_j}$ + cross-entropy; coherent probabilities).
- **Coefficients:** per-unit effects holding others fixed; standardize to compare; unstable under multicollinearity; associations, not causes.